Unity 2D Game Kit

The Explorer: 2D — 코드를 쓰지 않고 만드는 2D 플랫포머

튜토리얼 따라가기 · 스크립트 126개 읽기

원본 Unity Technologies, 2D Game Kit (Unity 2017.3+)

학습 과정 learn.unity.com/course/2d-game-kit

원광대학교 게임콘텐츠학과 「게임프로그래밍」

Copyright© by Wonkwang DC

이 자료의 구성

앞부분은 "따라 만들기", 뒷부분은 "코드 읽기"다

1부 — 따라 만들기 (3 ~ 14쪽)

Unity Learn의 Walkthrough 10단계를 그대로 따라간다.

타일맵으로 레벨을 그리고, 이동 플랫폼을 놓고, 발판으로 문을 열고, 적을 배치하고, 텔레포트를 연결한다.

이 과정에서 코드는 한 줄도 쓰지 않는다.

2부 — 코드 읽기 (15 ~ 33쪽)

그 "코드를 쓰지 않는" 동작들이 실제로 어떤 코드로 만들어져 있는지 연다.

126개 파일 · 11,552줄을 13개 그룹으로 나눠, 각 그룹의 핵심 구조를 그림과 코드로 함께 본다.

이 자료가 답하려는 질문 — "코드를 쓰지 않고 만든다"는 말은 무슨 뜻인가?

표기	뜻
굵은 남색	인스펙터에서 설정하는 항목. 코드를 고치지 않고 바꿀 수 있다
고정폭 글꼴	코드에 실제로 있는 클래스 · 함수 · 변수 이름. 철자가 정확하다
주황 상자	코드를 직접 읽어야만 알 수 있는 것. 문서에 없는 내용이다
빨간 글씨	설정을 잘못하면 동작하지 않거나 예외가 나는 지점

이 자료의 모든 수치와 이름은 실제 소스코드에서 확인한 것이다. 문서 권장값과 코드 기본값이 다를 때는 둘 다 적었다.

Copyright© by Wonkwang DC

2D Game Kit이란

아트 · 프리팹 · 스크립트가 함께 들어 있는 학습용 프로젝트



Ellen이 외계 행성에 불시착해 고대 문명의 폐허를 헤치고 지하 묘소에 이르는 이야기. Zone1~5 다섯 레벨과 보스가 들어 있다.

Copyright© by Wonkwang DC

학습 과정 — 5개 튜토리얼

learn.unity.com/course/2d-game-kit · Beginner · 4시간 25분

	튜토리얼	다루는 것
1	**Get Started with the 2D Game Kit**	설치 · Unity Hub · 에디터 화면 · ExampleScene 열기
2	**2D Game Kit Walkthrough**	10단계를 따라 한 레벨을 직접 만든다 — 이 자료의 1부
3	**Video: 2D Game Kit Walkthrough**	위 내용의 영상판. 45분, 8개 장
4	**2D Game Kit Reference Guide**	프리팹과 컴포넌트의 설정 항목 사전. 10개 절
5	**2D Game Kit Advanced Topics**	RandomAudioPlayer · VFXController · 데이터 영속화 · SceneLinkedSMB · 오브젝트 풀 · 행동 트리

4번과 5번이 이 자료의 2부와 겹친다

- Reference Guide는 "무엇을 어디에 넣는가"를 설명한다
- Advanced Topics는 "왜 그렇게 만들었는가"를 설명한다
- 이 자료는 거기에 "코드에서는 실제로 어떻게 되어 있는가"를 더한다

공식 문서가 말하지 않는 것

- 프리팹에 실제로 들어 있는 값과 코드의 초기값이 다르다
- 문서에 없는 클래스가 많다 (Gunner 보스, WaterArea, PlatformCatcher 등)
- 동작하지 않는 코드도 들어 있다

Unity 2017.3 이후 버전에서만 동작한다. 최신 Unity에서는 자동 업그레이드 과정을 거친다.

Copyright© by Wonkwang DC

설치와 프로젝트 구조

어디에 무엇이 들어 있는지부터 안다

설치하는 두 가지 길

Unity Hub의 Learn 탭

Hub > Learn 에서 2D Game Kit을 찾아 Download Project → Open Project.
프로젝트가 통째로 내려온다.

Asset Store

2D 프로젝트를 새로 만들고, 에셋 스토어에서 "2D Game Kit" (Unity Technologies)를 검색해 Import. 가져오는 데 8~10분 걸린다.

먼저 열 씬

`ExampleScene` — 학습용. 여기서 연습한다
`Start` — 게임 전체를 처음부터 플레이한다

Assets > 2DGameKit 아래

폴더	들어 있는 것
`Prefabs > Interactables`	MovingPlatform · Door · PressurePad · PushableBox · Key · Teleporter · 스위치
`Prefabs > Enemies`	Chomper · Spitter
`Prefabs > SceneControl`	TransitionStart · TransitionDestination · Checkpoint
`Art > Sprites > Environment`	배경 장식 스프라이트 (하위 폴더 많음)
`Scenes`	Start · Zone1 ~ Zone5 · Template
`Documentation`	키트에 함께 들어 있는 설명 문서

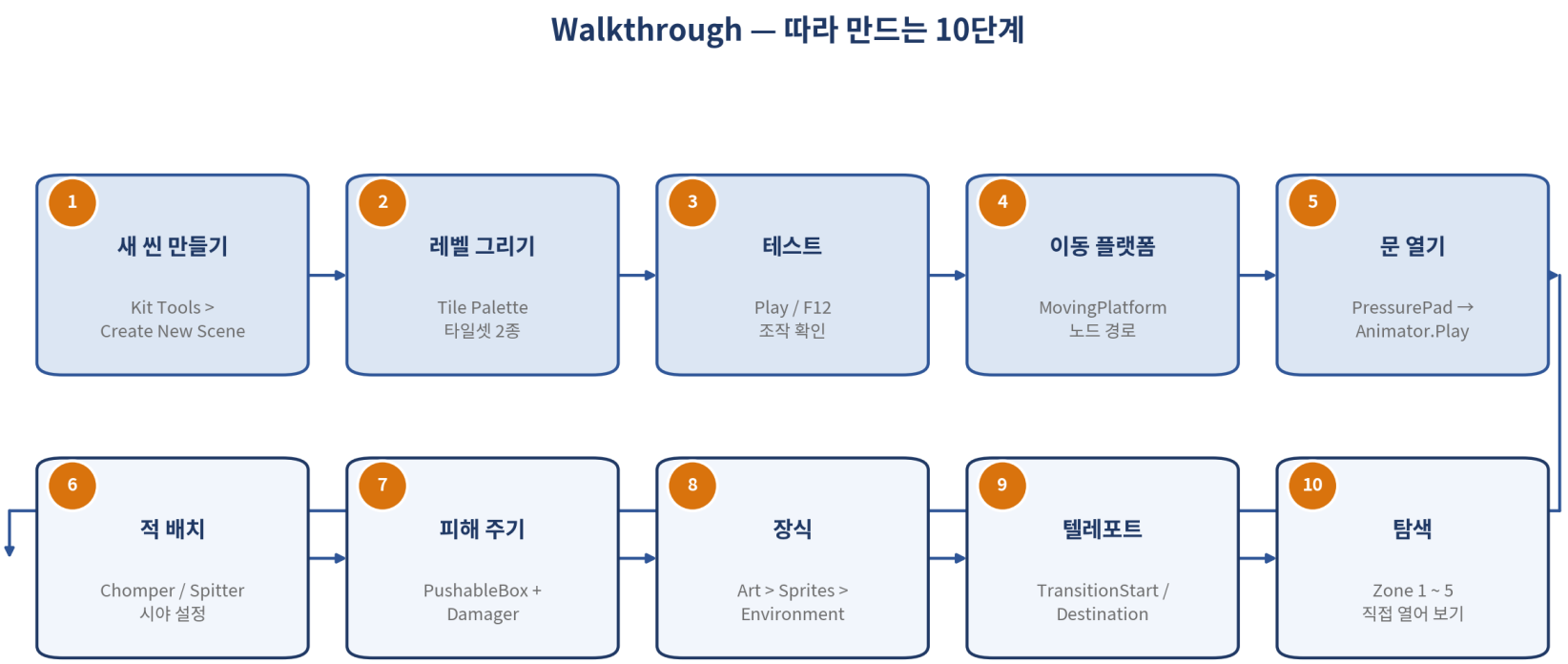
Zone1 ~ Zone5 를 직접 열어 보는 것이 가장 빠른 학습이다

완성된 레벨이 어떤 프리팹을 어떻게 배치하고, 어떤 이벤트를 어디에 연결해 두었는지 인스펙터에서 바로 볼 수 있다. 보스 Gunner는 Zone5에 있다.

Copyright© by Wonkwang DC

1부 — Walkthrough 10단계

코드를 한 줄도 쓰지 않고 한 레벨을 만든다



4~7단계가 핵심이다 — 모두 "인스펙터에서 이벤트를 연결하는" 같은 방식이다

다음 쪽부터 각 단계에서 실제로 무엇을 누르고 무엇을 설정하는지 본다.

Copyright© by Wonkwang DC

[1] 새 씬 만들기 · [2] 레벨 그리기

Kit Tools 메뉴와 Tile Palette

요구사항 1 새 씬 만들기 — Kit Tools > Create New Scene

	하는 일
①	상단 메뉴에서 Kit Tools 를 누른다
②	Create New Scene... 을 고르고 New Scene Name 에 이름을 적는다

새 씬에는 Ellen · 작은 발판 · 체력 UI · 카메라가 이미 놓여 있다. 바로 움직여 볼 수 있다.

요구사항 2 레벨 그리기 — Window > 2D > Tile Palette

	하는 일
①	Tile Palette 창을 열고 팔레트 메뉴에서 Tileset_Gamekit 을 고른다
②	두 가지 타일셋 중 하나를 고른다 — Grassy Rocks (바위) / Alien Structure (구조물)
③	팔레트에서 타일을 클릭한 뒤, 씬 뷰에서 왼쪽 버튼을 끌어 그린다
④	지우려면 Shift 를 누른 채 끌어 그린다. 화면 이동은 Q (손 도구)

주의 — 그리기 모드에서는 오브젝트를 옮길 수 없다

다음 단계에서 프리팹을 배치하려면 Tile Palette 창을 닫거나 이동 도구(W)를 먼저 선택해야 한다. 이것을 모르면 "왜 안 끌리지?"에서 한참 막힌다.

Copyright© by Wonkwang DC

[3] 테스트 — Ellen의 조작

코드에서 읽어낸 실제 바인딩

동작	키보드	게임패드	코드에서 일어나는 일
이동	A / D	왼쪽 스틱 좌우	maxSpeed 까지 groundAcceleration 으로 가속
점프	Space	buttonSouth	버튼을 일찍 떼면 상승 속도를 깎는 가변 점프
웅크리기	S	스틱 아래	Crouching 파라미터를 세운다
플랫폼 통과	S + Space	스틱 아래 + buttonSouth	밟은 바닥이 전부 PlatformEffector2D 일 때만. 0.5초 무적
근접 공격	K	buttonWest	바라보는 쪽으로 meleeAttackDashSpeed 만큼 미끄러진다
원거리 공격	O	buttonEast	누르고 있으면 shotsPerSecond 간격으로 연사
상호작용	E	buttonNorth	문 · 텔레포트 등에서 쓴다
일시정지	Escape	start	timeScale 을 0으로 하고 UIMenus 씬을 겹쳐 올린다

F12 — 무기 습득 여부를 켜고 끈다

플레이 중 F12를 누르면 근접 · 원거리 공격을 각각 켜고 끄는 메뉴가 나온다. 게임 안에서는 원래 아이템을 먹어야 얻는 능력이다.

제어권 설정과 무관하게 항상 동작하는 디버그 기능이다.

입력은 2단계로 막힌다

`Enable / Disable` — 그 동작을 습득했는가. 세이브에 저장된다

`GainControl / ReleaseControl` — 지금 조작할 수 있는가. 씬 전환 · 일시정지 중에 잠시 뺏는다

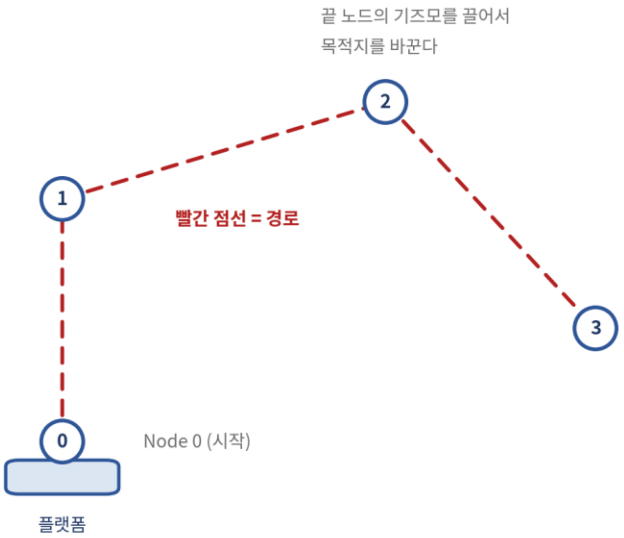
Copyright© by Wonkwang DC

[4] 이동 플랫폼 놓기

Prefabs > Interactables > MovingPlatform

MovingPlatform — 노드 경로와 3가지 반복 방식

씬 뷰에서 보이는 것



Add Node 버튼으로 노드를 늘린다.
노드마다 Wait Time(멈춰 있을 시간)을
따로 줄 수 있다.

Looping (MovingPlatformType)

BACK_FORTH

`0 - 1 - 2 - 3 - 2 - 1 - 0 - ...`

끝에 닿으면 역방향으로 되돌아온다

LOOP

`0 - 1 - 2 - 3 - 0 - 1 - ...`

마지막에서 첫 노드로 순환한다

ONCE

`0 - 1 - 2 - 3`

마지막 노드에 도착하면 그대로 멈춘다

코드 기본값

```
speed = 1.0f
platformType = BACK_FORTH
isMovingAtStart = true
```

튜토리얼 문서의
권장값은 Speed 2

플랫폼을 스위치로 제어하려면 — SingleUseSwitch 를 놓고 Interact on Trigger 2D 의 함수로 MovingPlatform > StartMoving() 을 고른 뒤, 플랫폼의 isMovingAtStart 체크를 푼다.

Copyright© by Wonkwang DC

[5] 발판으로 문 열기

이 단계가 키트 전체의 작동 원리다



이후의 거의 모든 것이 같은 방식이다

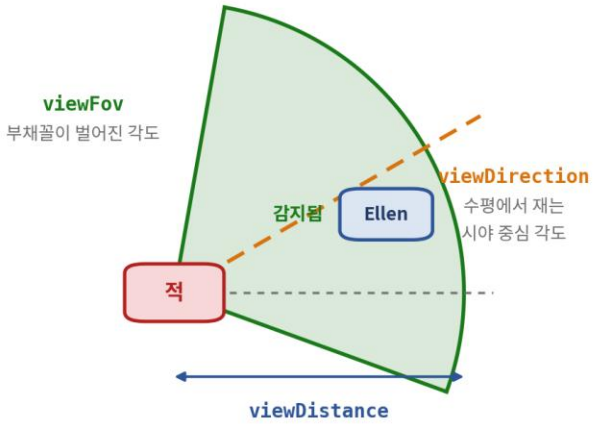
스위치, 열쇠, 체크포인트, 텔레포트, 적 소환기 — 전부 "어떤 조건이 되면 UnityEvent 를 쏜다"는 구조이고, 무엇을 할지는 인스펙터에서 정한다.

Copyright© by Wonkwang DC

[6] 적 배치하기

Prefabs > Enemies > Chomper / Spitter

적의 시야 판정 — EnemyBehaviour의 세 값



Chomper 와 Spitter 는 같은 컴포넌트다

	Chomper	Spitter
공격 방식	근접	원거리
채우는 필드	meleeDamager meleeRange attackDash	projectilePrefab
조종하는 것	애니메이터 SMB 4개	SpitterBT (행동 트리)

코드 기본값 주의

speed / viewFov / viewDistance 는 초기화가 없어 0이다.

프리팹에서 채우지 않으면
적이 움직이지도, 보지도 못한다.

gravity 10 · viewDirection 0 · timeBeforeTargetLost 3 · meleeRange 3

튜토리얼의 Spitter 예시값 : View Direction 30 View Fov 182 View Distance 6

씬 뷰에 초록 부채꼴로 시야가 그려진다. Ellen을 그 안에 넣어 보면서 값을 맞춘다.

Copyright© by Wonkwang DC

[7] 오브젝트로 피해 주기

아무 오브젝트에나 Damager 를 붙이면 흉기가 된다

	하는 일
①	PushableBox 프리팹을 Spitter 위쪽에 끌어다 놓는다
②	Add Component 에서 "Damage" 를 검색해 Damager 를 추가한다
③	씬 뷰에 초록 상자가 생긴다 — 이것이 피해 영역이다. Offset 과 Size 로 크기를 맞춘다
④	Hittable Layers 를 Enemy 로 설정한다 — 이걸 비워 두면 아무것도 맞히지 못한다
⑤	플레이해서 상자를 밀어 Spitter 위에 떨어뜨린다

```
// Damager.cs — 매 물리 프레임 사각형 영역을 검사한다
void FixedUpdate()
{
    if (!m_CanDamage) return;
    Vector2 scale  = m_DamagerTransform.lossyScale;
    Vector2 pointA = (Vector2)m_DamagerTransform.position
                   + scale * offset - scale * size * 0.5f;
    Vector2 pointB = pointA + scale * size;
    int hitCount = Physics2D.OverlapArea(pointA, pointB,
                       m_AttackContactFilter, m_AttackOverlapResults);
```

m_AttackContactFilter 는 Awake 에서 미리 만들어 둔다

Hittable Layers 는 필터의 layerMask 로, Can Hit Triggers 는 useTriggers 로 들어간다. 매 프레임 필터를 새로 만들지 않으므로 GC 할당이 생기지 않는다.

Copyright© by Wonkwang DC

[8] 장식 · [9] 텔레포트

레벨을 꾸미고 구역을 잇는다

요구사항 8 **장식 — Art > Sprites > Environment**

하위 폴더가 많으므로 화살표를 펼쳐 둘러본다. 완성된 Zone 씬에서 마음에 드는 스프라이트를 찾은 뒤 프로젝트 창에서 같은 것을 찾는 방식이 빠르다.

요구사항 9 **텔레포트 — Prefabs > SceneControl**

같은 씬 안에서 옮기기
TransitionStart 를 출발점에 놓는다
같은 프리팹을 하나 더 놓고 이름을 TransitionEnd 로 바꾼다
출발점의 Transition Point 에서 Transitioning Game Object = Ellen
`Transition Type = SameScene`
DestinationTransform 에 TransitionEnd 를 끌어 놓는다

다른 씬으로 보내기
도착 씬에 TransitionDestination 을 놓는다
Destination Tag 로 글자 하나를 고른다 (A~G)
그 씬을 Build Settings 에 등록한다
출발점의 `Transition Type = DifferentZone`
New Scene Name 과 Transition Destination Tag 를 도착 씬과 똑같이 맞춘다

가장 흔한 두 가지 실수

- 도착 씬을 Build Settings 에 등록하지 않았다 → "Scene couldn't be loaded" 오류
- Destination Tag 가 양쪽에서 다르다 → 전환은 되는데 Ellen이 엉뚱한 곳에 나타난다

Copyright© by Wonkwang DC

여기까지가 "코드 없이"

그런데 이 모든 동작은 어디에 쓰여 있나?

지금까지 한 일

타일을 그렸고, 프리팹을 끌어다 놓았고, 인스펙터에서 숫자를 바꾸고 함수를 꽂았다. 스크립트 파일은 한 번도 열지 않았다.
그런데 "발판에 올라가면 눌린다", "시야에 들어오면 쫓아온다", "맞으면 체력이 준다" 는 누군가 코드로 써 두어야 하는 동작이다.

"코드를 쓰지 않고 만든다" = 코드가 없다는 뜻이 아니라, 이미 쓰여 있다는 뜻

2부에서 읽을 것

1부에서 한 일	2부에서 여는 코드
Ellen을 움직였다	`PlayerCharacter` · `CharacterController2D` · `PlayerInput`
적이 쫓아왔다	`EnemyBehaviour` · `SceneLinkedSMB` · `BT` (행동 트리)
상자로 적을 눌렀다	`Damager` · `Damageable`
플랫폼에 올라탔다	`MovingPlatform` · `PlatformCatcher`
열쇠로 문을 열었다	`InventoryController` · `InventoryItem` · `HubDoor`
다른 구역으로 넘어갔다	`SceneController` · `IDataPersister` · `ScreenFader`
총을 쏘았다	`BulletPool` · `ObjectPool<,>` · `PoolObject<,>`

Copyright© by Wonkwang DC

2부 — 스크립트 전체 지도

126개를 13개 그룹으로 나눠 본다

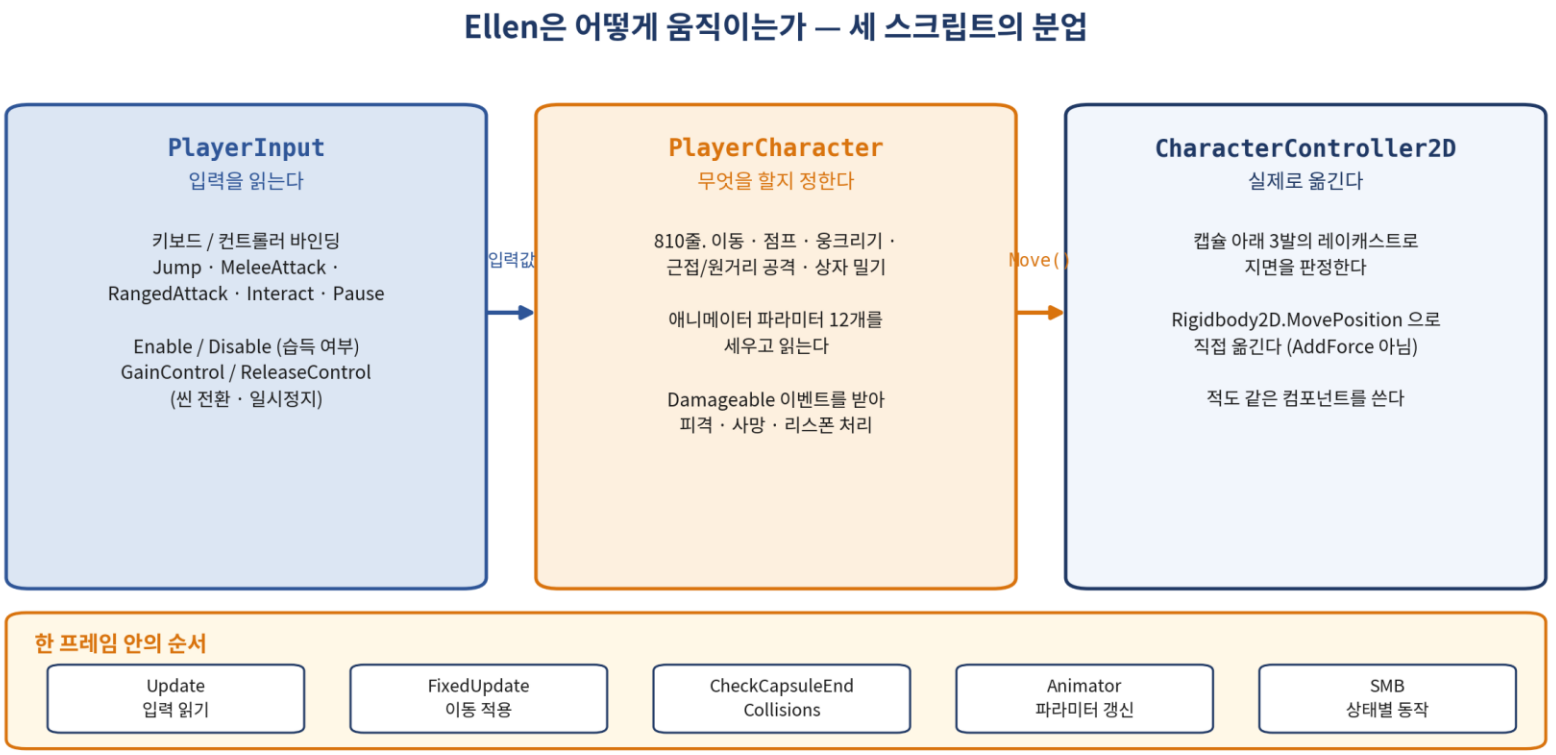


그룹 이름은 이 자료에서 쓰는 분류이자, 함께 만든 Doxygen 문서의 모듈 이름과 같다.

Copyright© by Wonkwang DC

Ellen은 어떻게 움직이는가

세 스크립트가 일을 나눠 맡는다



적(EnemyBehaviour)도 같은 CharacterController2D 를 쓴다. 이동 · 지면 판정은 플레이어와 적이 공유한다.

Copyright© by Wonkwang DC

PlayerCharacter 의 설정값

코드 기본값과 문서 권장값이 다르다

구분	필드	코드 기본값	문서 권장값	단위 · 뜻
이동	maxSpeed	**10**	**7**	유닛/초
	groundAcceleration	100	100	유닛/초²
	groundDeceleration	100	100	유닛/초²
	pushingSpeedProportion	0 (미초기화)	**0.5**	상자를 밀 때의 속도 비율
공중	gravity	**50**	**38**	유닛/초²
	jumpSpeed	**20**	**16.5**	유닛/초
	jumpAbortSpeedReduction	100	100	버튼을 뗐을 때 깎는 양
피격	hurtJumpAngle	45	45	도 (0.001 ~ 89.999)
	hurtJumpSpeed	**5**	**8**	유닛/초
	flickeringDuration	**0.1**	**0.05**	초 — 깜빡임 주기
공격	meleeAttackDashSpeed	**5**	**15**	유닛/초
	shotsPerSecond	**1**	**2**	발/초
	bulletSpeed	**5**	**30**	유닛/초

왜 다른가 — 코드의 초기값은 "스크립트를 새로 붙였을 때의 값"이고, 프리팹에는 그와 다른 값이 저장되어 있다

Ellen 프리팹을 인스펙터로 열면 오른쪽 열의 값이 보인다. 코드만 읽으면 "점프가 20이구나" 하고 잘못 이해하게 된다. 반대로 인스펙터만 보면 미초기화 필드가 어떤 것인지 알 수 없다.

Copyright© by Wonkwang DC

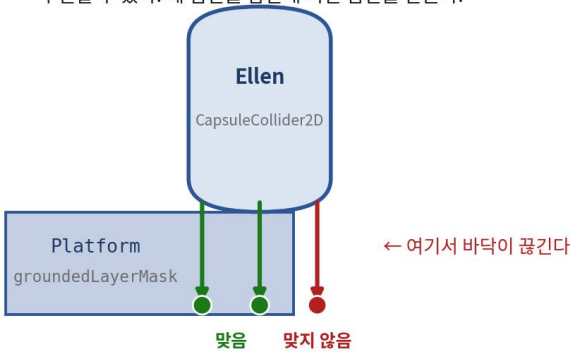
지면 판정

Rigidbody2D 충돌 콜백을 쓰지 않는다

지면 판정 — 왜 충돌 콜백이 아니라 레이캐스트 3발인가

발 한쪽만 턱에 걸린 상황

3발을 따로 쏘기 때문에 "완전히 떠 있다"와
구별할 수 있다. 세 법선을 합산해 지면 법선을 만든다.



raycast 사거리

```
m_Capsule.size.x * 0.5f + groundedRaycastDistance * 2f
```

groundedRaycastDistance 기본 0.1

왜 OnCollisionEnter2D 가 아닌가

충돌 콜백은 "닿았다 / 떨어졌다"는 사건만 준다.
점프 · 착지 · 공중 제어에는 매 프레임
"지금 땅에 있는가"라는 상태 값이 필요하다.

레이캐스트는 groundedLayerMask 로
무엇을 땅으로 볼지 코드가 직접 정한다.

올라가는 중은 접지가 아니다

```
IsGrounded = relativeVelocity.y <= 0f
```

relativeVelocity 는 MovingPlatform 속도를
뺀 상대 속도다. 그래서 움직이는 플랫폼에
실려 가도 접지가 유지된다.

같은 함수에 bottom = false 를 주면 천장 판정이 된다. 한쪽에서만 통과하는 플랫폼은 천장에서 제외한다.

Copyright© by Wonkwang DC

애니메이터 파라미터 12개

코드와 애니메이터가 만나는 지점

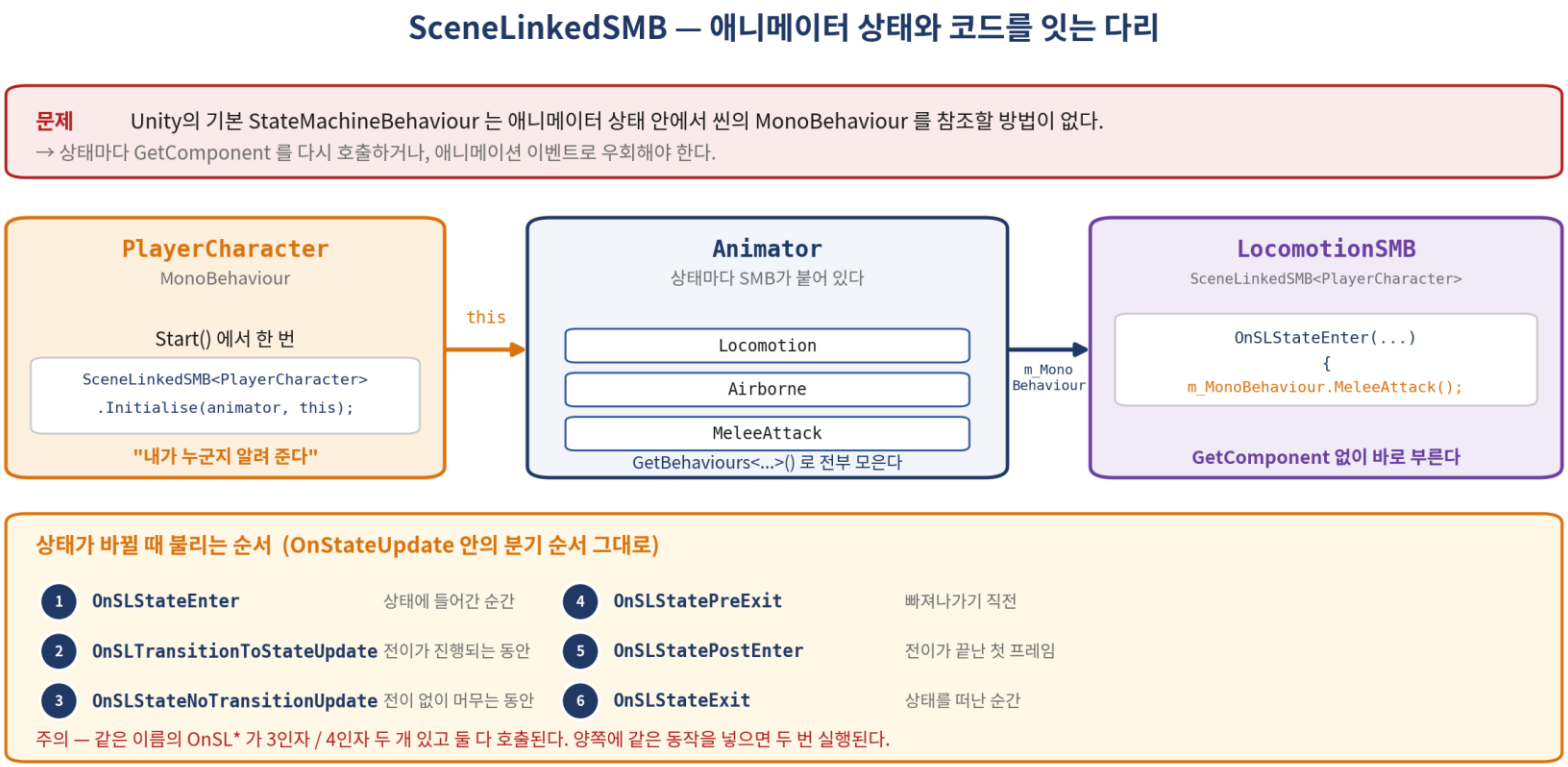
```
// 문자열 비교 비용을 없애려고 해시를 미리 계산해 둔다
readonly int m_HashHorizontalSpeedPara = Animator.StringToHash("HorizontalSpeed");
readonly int m_HashGroundedPara        = Animator.StringToHash("Grounded");
...
m_Animator.SetFloat(m_HashHorizontalSpeedPara, m_MoveVector.x);   // 매 FixedUpdate
```

파라미터	종류	누가 세우나	쓰임
HorizontalSpeed	float	FixedUpdate	걷기 · 달리기 전환
VerticalSpeed	float	FixedUpdate	상승 · 하강 구분
Grounded	bool	CheckForGrounded()	지상 ↔ 공중 전환
Crouching	bool	CheckForCrouching()	웅크리기 상태
Pushing	bool	CheckForPushing()	상자 미는 상태
HoldingGun	bool	CheckForHoldingGun()	총 든 자세 유지
MeleeAttack	trigger	MeleeAttack()	근접 공격 시작
Hurt	trigger	OnHurt()	피격. Respawn()이 지운다
ForcedRespawn	trigger	OnHurt()	체력이 남았는데 강제 리스폰
Dead	trigger	OnDie()	사망
Respawn	trigger	Respawn()	부활
Timeout	—	(없음)	해시만 선언되고 쓰이는 곳이 없다

Copyright© by Wonkwang DC

SceneLinkedSMB

애니메이터 상태에서 MonoBehaviour 를 바로 부른다



공식 Advanced Topics 에서 한 절을 할애해 설명하는 구조다. 이 키트에서 가장 재사용하기 좋은 코드이기도 하다.

Copyright© by Wonkwang DC

Ellen의 상태와 SMB 대응

애니메이터 상태마다 담당 스크립트가 따로 있다

애니메이터 상태	SMB 클래스	그 상태에서 하는 일
지상 이동	`LocomotionSMB`	콜라이더 하단 정렬 · 지상 이동 · 웅크리기/착지/밀기/사격 입력 판정
지상 이동 (총)	`LocomotionWithGunSMB`	위와 같고, 퇴장 시 다음 상태가 WithGun 태그가 아니면 총을 내린다
공중	`AirborneSMB`	UpdateJump · 공중 수평/수직 이동 · CheckForGrounded
공중 (총)	`AirborneWithGunSMB`	위와 같고 총 내리기 판정이 더 있다
웅크리기	`CrouchSMB`	입력 없는 지상 이동. 아래 + 점프 시 MakePlatformFallthrough
웅크리기 (총)	`CrouchWithGunSMB`	위와 같고 CheckAndFireGun 이 더 있다
근접 공격	`MeleeAttackSMB`	EnableMeleeAttack + 대시. 공중으로 떨어지면 같은 진행도로 공중 공격을 재생
`AirborneMeleeAttack`	`AirborneMeleeAttackSMB`	dashWhileAirborne 일 때만 대시한다
밀기	`PushingSMB`	pushingSpeedProportion 배 느린 이동 + MovePushable
피격	`HurtSMB`	hurtJumpSpeed 만큼 넉백 + StartFlickering
사망 / 사망 착지	`DeadSMB` / `DeadLandingSMB`	넉백 속도를 주고 착지까지 중력 적용, 착지 뒤 감속
리스폰	`RespawnSMB`	SetMoveVector(Vector2.zero) — 잔여 속도를 지운다

WithGun 계열 3개는 애니메이터 상태의 태그가 "WithGun" 인지 코드에서 확인한다. AirborneMeleeAttack 만 코드에 상태 이름이 직접 적혀 있다.

주의 — 같은 이름의 OnSL* 가 3인자/4인자 두 개 있고 둘 다 호출된다. 양쪽에 같은 동작을 넣으면 두 번 실행된다.

Copyright© by Wonkwang DC

피해 시스템

이 키트의 전투는 전부 이 한 쌍으로 이루어진다

피해 처리 — Damager 가 Damageable 을 때린다

Damager
"때리는 쪽"

`TakeDamage(this, ignoreInvincibility)`

Ellen의 칼 · 총알 · 적의 몸 · 산성 웅덩이

Damageable
"맞는 쪽"

1	`Damager.FixedUpdate()`	m_CanDamage 가 false면 바로 끝낸다
2	공격 영역 계산	offset·size 에 lossyScale 을 곱한다. offsetBasedOnSpriteFacing 이면 바라보는 쪽으로 offset.x 부호를 뒤집는다
3	`Physics2D.OverlapArea(pointA, pointB, filter, results)`	filter 는 Awake 에서 hittableLayers 와 canHitTriggers 로 미리 만들어 둔다
4	맞은 콜라이더마다 GetComponent<Damageable>()	
5	Damageable 이 없으면 → OnNonDamageableHit	총알이 벽에 맞았을 때 이펙트를 내는 경로
6	Damageable 이 있으면 → OnDamageableHit	피해를 주기 직전에 발생한다
7	`Damageable.TakeDamage()`	무적이고 ignoreInvincible 이 아니거나 체력이 0이면 아무 일도 없다
8	체력 감소 → OnHealthSet	HealthUI 가 이 이벤트를 받아 하트를 끈다
9	넉백 방향 계산 → OnTakeDamage	EnemyBehaviour.Hit() / PlayerCharacter.OnHurt() forceRespawn 은 여기서 읽혀 강제 리스폰이 된다
10	체력이 0이면 → OnDie	EnemyBehaviour.Die(). 무적을 켜고 disableOnDeath 면 SetActive(false)
11	disableDamageAfterHit 이면 DisableDamage()	한 모션에 한 번만 맞게 한다. 다음 FixedUpdate 부터 적용

무적 시간은 어떻게 만드나
EnableInvulnerability() 가 타이머에 invulnerabilityDuration 을 넣고, Update() 가 매 프레임 Time.deltaTime 만큼 깎는다.

이벤트는 코드가 연결하지 않는다
PlayerCharacter.OnHurt / OnDie 는 코드에서 부르는 곳이 없다.
인스펙터에서 Damageable 의 이벤트에 꽂아야 동작한다.

Copyright© by Wonkwang DC

적 — 한 컴포넌트, 두 가지 적

Chomper와 Spitter는 EnemyBehaviour 를 공유한다

	Chomper (근접)	Spitter (원거리)

채우는 필드	`meleeDamager`
	`meleeRange`
	`attackDash`
	`attackForce`
		`projectilePrefab`
		`shootingOrigin`
		`shootAngle · shootForce`
		`fireRate`

조종하는 것	**애니메이터 SMB 4개**	**SpitterBT — 행동 트리**
행동 전환	애니메이터 상태 전이 조건으로 순찰 → 추격 → 공격 → 사망	BT 노드가 매 프레임 tick 되어 Success / Failure / Continue 를 낸다

Chomper 전용 SMB 4개

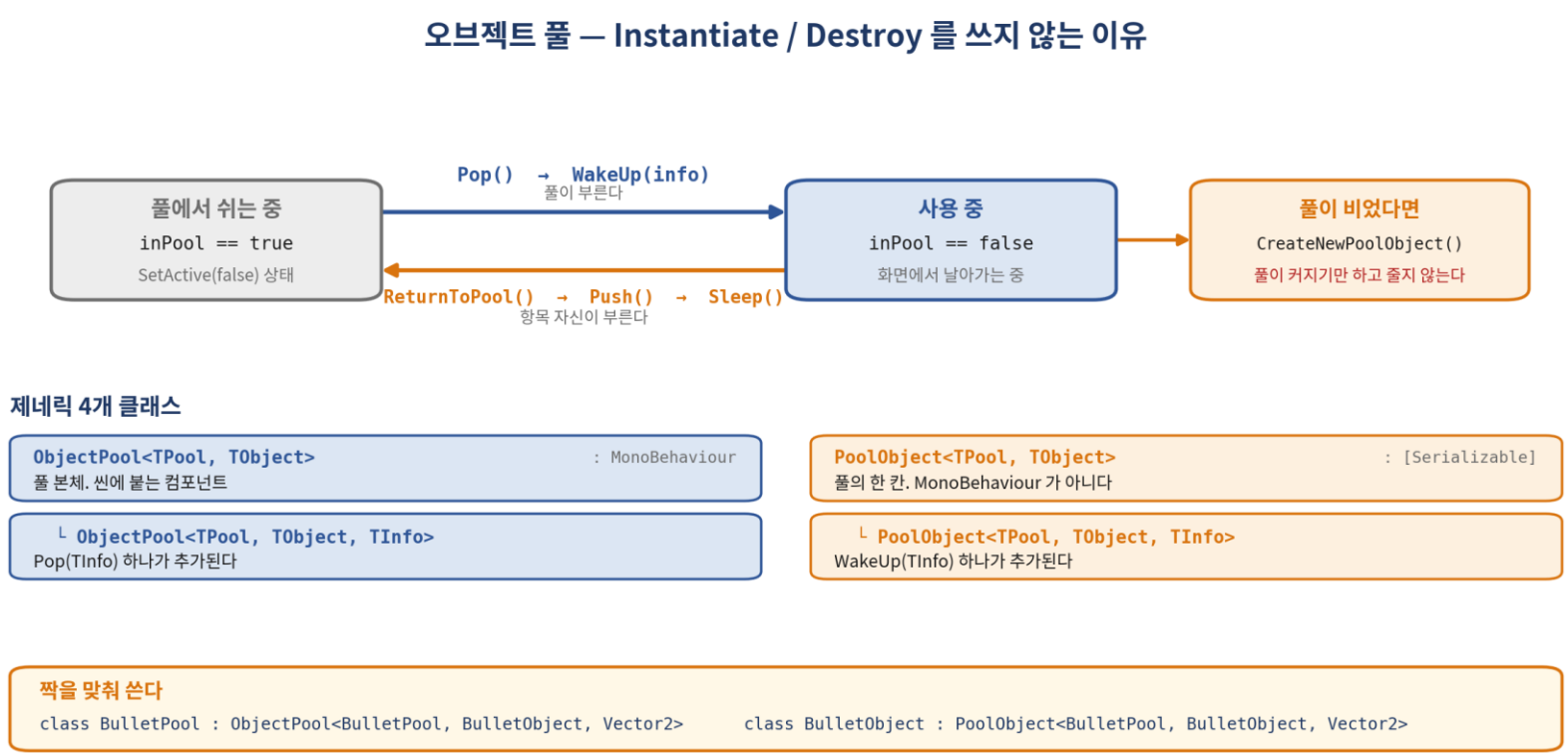
SMB	하는 일
`ChomperPatrolSMB`	CheckForObstacle 로 앞을 보고 막혔으면 되돌아선다. 매 프레임 ScanForPlayer 로 탐색
`ChomperRunToTargetSMB`	speed * 2 로 달린다. 앞에 장애물이 있으면 ForgetTarget 으로 추격을 포기한다
`ChomperAttackSMB`	퇴장 시 수평 속도를 0으로. 공격 판정은 애니메이션 이벤트가 담당한다
`ChomperDeathSMB`	진입 시 피해를 모두 끄고, 퇴장 시 SetActive(false) — Destroy 가 아니라 재사용을 위해

같은 CheckForObstacle 을 순찰은 "되돌아서는 신호"로, 추격은 "포기하는 신호"로 쓴다 — 상태별로 해석이 다르다

Copyright© by Wonkwang DC

오브젝트 풀

Instantiate / Destroy 를 쓰지 않는 이유



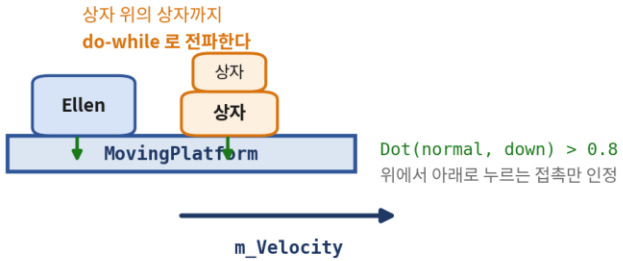
총알(BulletPool) · 적 소환기(EnemySpawner) · 이펙트(VFXController) 가 모두 이 구조를 쓴다.

Copyright© by Wonkwang DC

PlatformCatcher

움직이는 플랫폼 위에서 걷고 점프할 수 있는 이유

PlatformCatcher — 올라탄 것을 함께 옮기기



매 FixedUpdate 마다

1. inContact / checkedThisFrame 를 모두 false 로 지운다
2. Rigidbody2D.GetContacts 로 접촉점을 미리 만든 배열에 받는다
3. 위에서 누르는 접촉이면 CaughtObject 목록에 넣는다
4. 이미 접촉 중인 것에 대해 한 번 더 조사해 위에 쌓인 것까지 찾는다
5. 끊긴 접촉은 bounds 로 보정한다 (y 간격 0.05 이내)
6. MovingPlatform 이 MovePosition 한 직후 MoveCaughtObjects(velocity)
7. 캐릭터는 CharacterController2D.Move, 나머지는 MovePosition

왜 플랫폼의 자식(parent)으로 붙이지 않는가

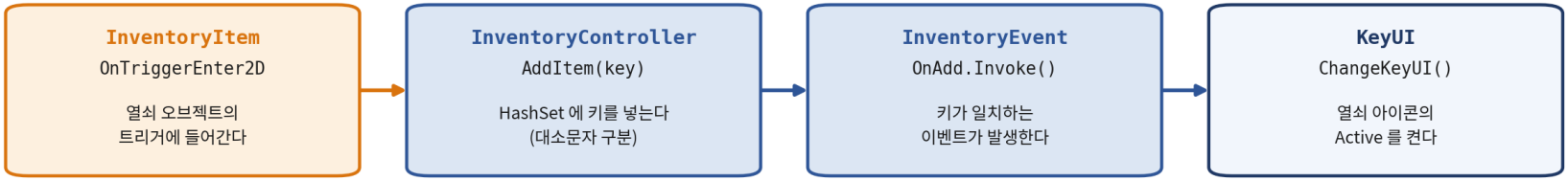
자식으로 붙이면 운반되는 쪽의 Transform 이 플랫폼에 종속되어, 자기 물리 시뮬레이션과 입력 이동이 뒤틀린다.
접촉으로 찾아내 같은 변위만큼 직접 옮기면, 운반되는 쪽은 자기 이동을 그대로 유지한 채 플랫폼 변위만 더해 받는다.
그래서 움직이는 플랫폼 위에서 걷고, 점프하고, 상자를 미는 것이 모두 자연스럽게 된다.

Copyright© by Wonkwang DC

인벤토리와 문

열쇠를 먹고 문을 여는 전 과정

열쇠를 먹으면 문이 열린다 — 그 사이에 일어나는 일



문을 여는 두 가지 경로

A — 트리거형 (문 앞에 다가갔을 때 검사)

InteractOnTrigger2D 의 inventoryChecks 가
inventoryItems 를 앞에서부터 훑는다.
전부 가지면 OnHasItem, 없는 키를 처음 만나면
OnDoesNotHaveItem 을 쏘고 즉시 멈춘다 (AND 조건).

B — 상태형 (HubDoor, 모은 수에 따라 모습이 바뀜)

Update() 가 매 프레임 키 개수를 세어
unlockStateSprites[보유 수 - 1] 로 교체한다.
전부 모으면 onUnlocked 발생 — 매 프레임 검사하므로
다 모은 뒤에는 프레임마다 다시 발생한다.

씬을 넘어가도 유지되는 이유

InventoryController 와 InventoryItem 이 둘 다 IDataPersister 다. 전자는 보유 키 전체를, 후자는 "이미 먹었는지"를 저장한다.

키 문자열은 대소문자를 구분한다

네 곳의 철자가 모두 정확히 같아야 한다 — 하나라도 다르면 "먹었는데 아무 일도 안 일어난다".

`InventoryItem.inventoryKey` `InventoryEvent.key` `InventoryChecker.inventoryItems` `KeyUI.keyNames`

Copyright© by Wonkwang DC

씬 전환과 데이터 영속화

구역을 넘어가도 진행 상황이 유지되는 구조

씬 전환 — SceneController.Transition 코루틴의 14단계

1. **`m_Transitioning = true`**
   중복 발동을 막는 플래그
2. **`PersistentDataManager.SaveAllData()`**
   데이터 저장 — 페이드보다 먼저다
3. **`PlayerInput.ReleaseControl()`**
   조작을 뺏는다
4. **`ScreenFader.FadeSceneOut(Loading)`**
   화면이 어두워질 때까지 기다린다
5. **`PersistentDataManager.ClearPersisters()`**
   구현체 목록만 비운다. 저장소는 남는다
6. **`SceneManager.LoadSceneAsync(newSceneName)`**
   새 씬 로드 완료까지 기다린다
7. **새 씬의 PlayerInput 을 찾아 ReleaseControl()**
8. **`PersistentDataManager.LoadAllData()`**
   예약만 한다 — 실제 LoadData 는 다음 Update 에
9. **`GetDestination(destinationTag)`**
   도착점을 태그로 찾는다 (A~G)
10. **`SetEnteringGameObjectLocation(entrance)`**
    플레이어를 도착점으로 옮긴다
11. **`SetupNewScene(transitionType, entrance)`**
    DifferentZone 일 때만 구역 재시작 지점 갱신
12. **`entrance.OnReachDestination.Invoke()`**
    도착 이벤트. CharacterStateSetter 를 꽂는 자리
13. **`ScreenFader.FadeSceneIn()`**
    화면이 밝아질 때까지 기다린다
14. **`GainControl()` / `m_Transitioning = false`**
    조작을 돌려준다

저장이 먼저, 페이드가 나중

흔히 "페이드 → 저장"을 예상하지만 코드는 반대다.
아직 이전 씬이 살아 있을 때 저장해야
오브젝트들의 상태를 읽을 수 있기 때문이다.

로드는 한 프레임 뒤에 반영된다

LoadAllData() 는 schedule 에 쌓아 두고
다음 Update() 에서 실행한다. "목록을 돌면서
목록을 수정"하는 예외를 피하기 위해서다.

IDataPersister 를 구현하는 8개 클래스 — 이들의 상태만 씬을 넘어 유지된다

- `PlayerInput`
- `Damageable`
- `EnemySpawner`
- `WeaponPickup`
- `HubDoor`
- `InventoryItem`
- `InventoryController`
- `DirectorTrigger`

PersistenceType : DoNotPersist · ReadOnly · WriteOnly · ReadWrite dataTag 는 씬 안에서 고유해야 한다

Copyright© by Wonkwang DC

행동 트리 (Behaviour Tree)

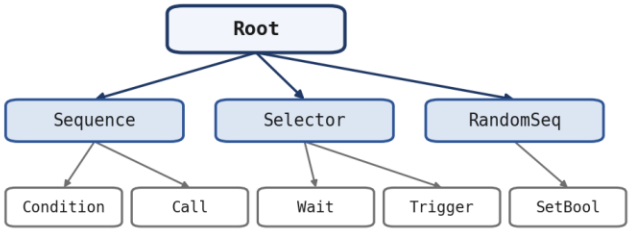
에디터 없이 코드로만 쓰는 AI 구조

행동 트리 (BT) — 매 프레임 tick 되어 세 값 중 하나를 돌려준다

Success	Failure	Continue	Abort
성공하고 끝났다	실패하고 끝났다	아직 진행 중이다	트리 전체를 중단한다

Continue 가 있기 때문에 트리는 한 프레임에 끝나지 않고 틱마다 조금씩 앞으로 나아간다.

구조 노드



말단 노드 — 실제로 무언가 하는 쪽

노드별 반환 규칙

Sequence	자식이 Success면 다음 자식으로. Failure면 인덱스를 0으로 되돌리고 Failure
Selector	자식이 Success면 즉시 Success. Failure면 다음 자식으로
Block	자식의 Failure / Abort를 무시하고 다음으로. 끝까지 돌면 Success
Root	while 루프로 Continue가 날 때까지 연달아 처리한다
Repeat(n)	n회를 채우면 Success. 카운트를 0으로 되돌려 재사용할 수 있다
RandomSequence	누적 가중치로 자식 하나를 추첨한다
While	조건이 true인 동안 항상 Continue. false가 되면 Failure

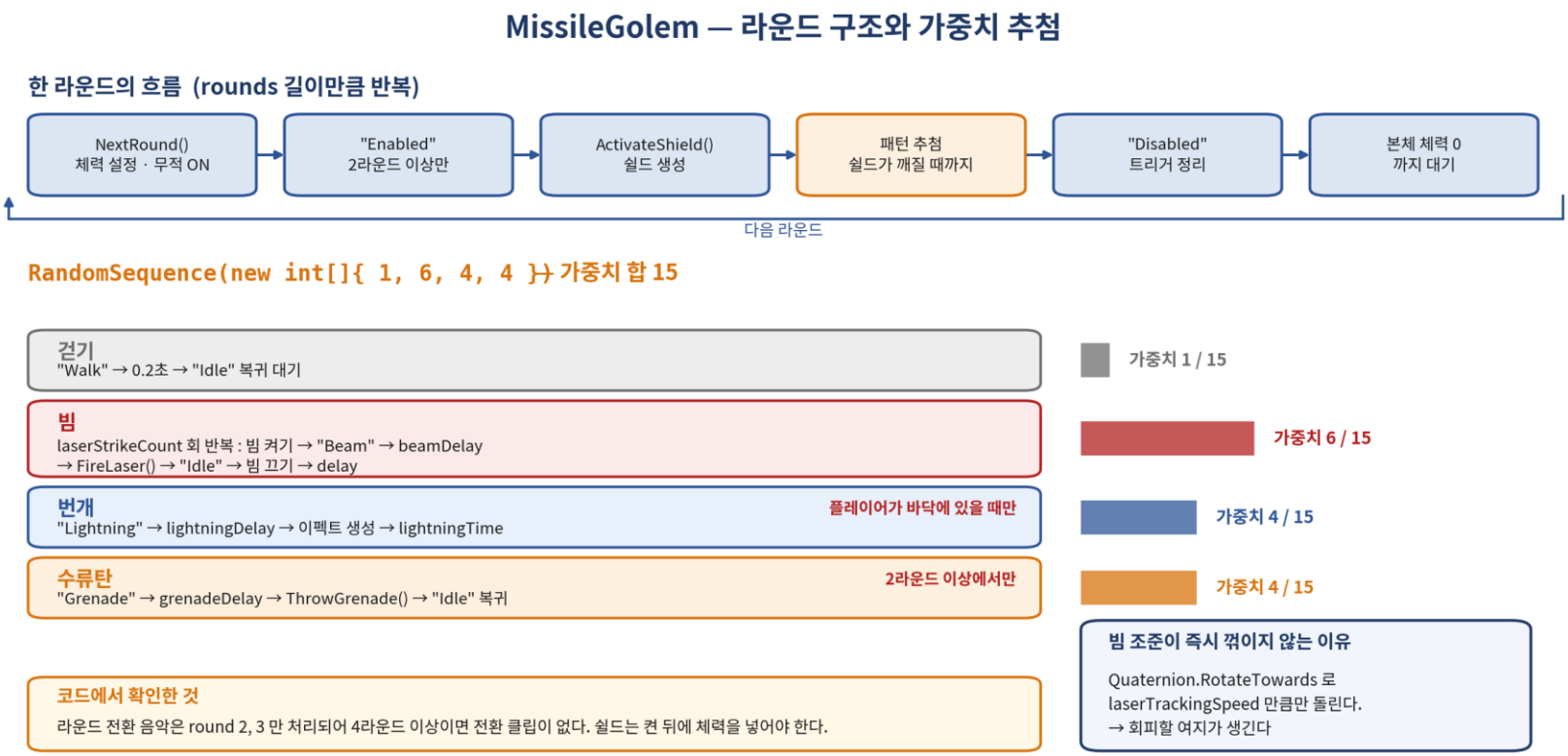
코드를 읽어야 알 수 있는 것

공식 문서에 없는 네 번째 값	BTState 에는 Abort 가 있다. Terminate 노드가 이것을 낸다
While 은 Success 를 내지 않는다	조건이 true면 자식 결과와 무관하게 Continue, false면 Failure뿐이다
Selector 의 shuffle 은 동작하지 않는다	생성자 시점에 children 이 비어 있어 아무것도 섞지 않는다
Root 는 즉시 노드를 몰아서 실행한다	Call / Trigger 같은 노드는 한 프레임에 연달아 처리된다

Copyright© by Wonkwang DC

MissileGolem 보스

행동 트리로 만든 라운드제 보스전



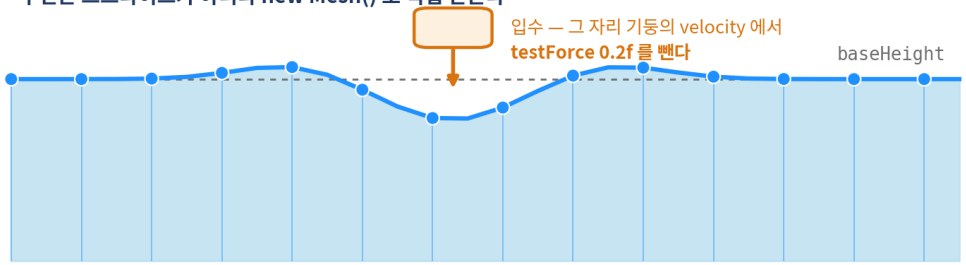
Copyright© by Wonkwang DC

WaterArea

수면을 코드로 만들고 물리로 출렁인다

WaterArea — 수면을 코드로 만들고 1차원 스프링으로 출렁인다

수면은 스프라이트가 아니라 new Mesh() 로 직접 만든다



WaterColumn 하나 = 정점 하나

```
count = CeilToInt(size.x * (pointPerUnits - 1)) + 1
```

`tension`	기준 높이로 되돌리는 복원력 (훅의 법칙)
`dampening`	속도에 곱하는 감쇠. 0~1 사이
`neighbourTransfer`	이웃 기둥으로 높이 차를 퍼뜨리는 확산
`pointPerUnits`	1 유닛당 분할 점 수 — 물결의 해상도

매 프레임의 계산

```
left  = neighbourTransfer * (h[i-1] - h[i])
right = neighbourTransfer * (h[i+1] - h[i])
force = left + right
      + tension * (baseHeight - h[i])

velocity = dampening * velocity + force
h[i]    += velocity
```

이웃과의 높이 차를 퍼뜨리고 기준 높이로 당긴다

코드를 읽어야 알 수 있는 것

Time.deltaTime 을 곱하지 않는다.
→ 물결 속도가 프레임 수에 의존한다.

피해는 WaterArea 가 주지 않는다.
같은 오브젝트의 Damager 가 준다.
WaterArea 는 그 Damager 의 size·offset 을
물 영역과 같게 맞춰 줄 뿐이다.

454줄로 이 키트에서 가장 긴 연출 스크립트다. 물리 시뮬레이션을 직접 짜 보는 예제로 읽을 만하다.

Copyright© by Wonkwang DC

에디터 확장

레벨 디자인을 쉽게 만드는 17개 스크립트

스크립트	무엇을 바꾸나
MovingPlatformEditor	씬 뷰에 플랫폼 경로의 노드 핸들을 그려 직접 끌어 옮기게 한다. 이 키트에서 가장 쓸모 있는 에디터 확장이다
PlayerCharacterEditor	Ellen의 수많은 필드를 Movement / Airborne / Hurt / Melee / Ranged / Audio / References 그룹으로 묶어 보여 준다
DamagerEditor	공격 범위를 씬 뷰에서 박스 핸들로 끌어 조절하게 한다 (초록 상자)
SceneNameDrawer	씬 이름을 직접 타이핑하지 않고 드롭다운에서 고르게 한다. Build Settings 에 없는 씬 이름을 적는 실수를 막는다
PressurePadEditor	활성 조건(개수 / 질량)에 따라 필요한 필드만 보여 준다

Editor 폴더의 규칙

이름이 "Editor" 인 폴더 안의 스크립트는 빌드에 포함되지 않는다. UnityEditor 네임스페이스를 쓰는 코드는 반드시 여기 넣어야 빌드가 된다.

동작하지 않는 에디터 코드

ParameterSetterDrawer 는 바깥 클래스의 animator 필드를 찾으려다 null 을 받는다. 그러면 항목 높이가 0이 되어 인스펙터에 아무것도 그려지지 않는다.

에디터 확장은 "코드를 쓰지 않고 만든다"를 가능하게 하는 또 하나의 축이다. 인스펙터가 쓰기 편해야 코드를 안 열게 된다.

Copyright© by Wonkwang DC

코드를 읽다 발견한 것들

공식 문서에 없는 내용이다

위치	내용
SavedData.cs	죽은 코드다. Dictionary 5개가 초기화되지 않아 호출하면 예외가 나고, 프로젝트 전체에서 호출하는 곳이 0건이다
BTAI.While	Success 를 내는 경로가 없다. 조건이 true면 Continue, false면 Failure 뿐이다
BT.Selector	생성자의 shuffle 인자는 그 시점에 children 이 비어 있어 아무것도 섞지 않는다
BTState	공식 문서는 Success / Failure / Continue 세 가지만 말하지만 Abort 가 하나 더 있다
PlayerCharacter	OnHurt / OnDie 를 코드에서 부르는 곳이 없다. 인스펙터 연결이 없으면 피격이 동작하지 않는다
HubDoor.Update	매 프레임 CheckInventory 를 돌리므로 키를 다 모은 뒤에는 onUnlocked 가 계속 발생한다
WaterArea	물결 계산에 Time.deltaTime 을 곱하지 않는다 — 프레임 수에 따라 물결 속도가 달라진다
MissileGolem	라운드 전환 음악이 round 2, 3 만 처리되어 4라운드 이상이면 전환 클립이 없다
GunnerBeamController	콜백이 전부 주석 처리된 빈 StateMachineBehaviour 다
EnemySpawner.Start	실제 소환 수가 아니라 concurrentEnemiesToBeSpawned 를 누적 카운트에 더한다
PriorityQueue	SiftDown 이 이름과 반대로 원소를 루트 쪽으로 올린다. Contains 는 O(n) 선형 탐색
InputDisplayUI	F12 토글 UI가 아니다. 현재 바인딩을 읽어 조작법 안내 문구를 만드는 UI다

이것이 "남의 코드를 읽는다"는 일의 실제 모습이다

공식 학습 자료라고 해서 모든 코드가 완벽하지는 않다. 읽으면서 "왜 이렇게 했지?"를 묻고, 답이 없으면 결함일 수도 있다고 의심하는 것이 코드를 읽는 능력이다.

Copyright© by Wonkwang DC

이 키트에서 배울 설계 원칙

내 게임에 그대로 쓸 수 있는 것들

1 **동작은 코드에, 연결은 인스펙터에**

"무엇이 일어나는가"는 UnityEvent 로 열어 두고, "무엇과 이어지는가"는 씬에 저장한다. 같은 발판 스크립트를 문 열기에도, 플랫폼 출발에도 쓸 수 있는 이유다.

2 **역할이 다르면 스크립트를 나눈다**

PlayerInput(읽기) · PlayerCharacter(판단) · CharacterController2D(이동). 그래서 적이 CharacterController2D 만 떼어다 쓸 수 있다.

3 **상태는 애니메이터가, 동작은 코드가**

SceneLinkedSMB 가 둘을 잇는다. 상태 전이 조건은 애니메이터 창에서 눈으로 보고 고치고, 각 상태에서 할 일은 작은 스크립트 하나에 적는다.

4 **만들고 버리지 않고 재사용한다**

ObjectPool / PoolObject 한 쌍으로 총알 · 적 · 이펙트를 모두 처리한다. Instantiate 와 Destroy 를 반복하면 프레임이 끊긴다.

5 **"없을 때"를 코드가 정해 둔다**

Damager 의 hittableLayers 가 비면 아무것도 못 맞히고, EnemyBehaviour 의 speed 가 0이면 적이 서 있는다. 기본값이 곧 설계다.

Copyright© by Wonkwang DC

총정리 — 무엇을 어디서 보나

필요할 때 이 표로 돌아온다

하고 싶은 일	씬에서	코드에서
주인공 조작을 바꾸고 싶다	Ellen 프리팹의 인스펙터	`PlayerCharacter.cs · PlayerInput.cs`
점프감을 다듬고 싶다	gravity · jumpSpeed · jumpAbortSpeedReduction	`PlayerCharacter.UpdateJump()`
적을 새로 만들고 싶다	Chomper 프리팹을 복제해 수정	`EnemyBehaviour.cs + SMB 또는 BT`
새 함정을 만들고 싶다	아무 오브젝트에 Damager 추가	`Damager.cs — 건드릴 필요 없다`
문을 새 조건으로 열고 싶다	InteractOnTrigger2D 의 inventoryChecks	`InventoryChecker (InventoryController 안)`
움직이는 발판을 만들고 싶다	MovingPlatform 프리팹 + 노드 편집	`MovingPlatform.cs · PlatformCatcher.cs`
구역을 새로 잇고 싶다	TransitionStart / Destination + Build Settings	`SceneController.Transition()`
진행 상황을 저장하고 싶다	해당 컴포넌트의 Data Settings	`IDataPersister 구현`
총알·이펙트를 추가하고 싶다	프리팹과 풀 크기 설정	`ObjectPool<,> + PoolObject<,>`
발소리를 바닥마다 다르게	RandomAudioPlayer 의 타일 오버라이드	`RandomAudioPlayer.cs · AudioSurface.cs`

함께 만든 자료

- Doxygen 주석 소스 — 126개 .cs 파일 전체에 한국어 주석을 달았다. 원본 코드는 한 글자도 바꾸지 않았다
- Doxygen HTML 문서 — 클래스 다이어그램과 소스 브라우저가 들어 있다. html/index.html 을 연다
- Game Design Document — 이 키트를 하나의 게임으로 보고 GDD 양식에 맞춰 정리했다

Copyright© by Wonkwang DC

스스로 확인하기

답이 바로 떠오르지 않으면 해당 쪽으로 돌아간다

1. "코드를 쓰지 않고 만든다"는 말이 정확히 무슨 뜻인가? — 3 · 14 쪽
2. 발판을 밟으면 문이 열린다. 이 연결은 어디에 저장되어 있나? — 10 쪽
3. Ellen의 점프 속도를 바꾸려면 코드와 프리팹 중 어디를 고쳐야 하나? 왜인가? — 17 쪽
4. 왜 지면 판정에 OnCollisionEnter2D 를 쓰지 않고 레이캐스트를 세 발 쏘나? — 18 쪽
5. SceneLinkedSMB 가 없으면 애니메이터 상태에서 무엇이 불편해지나? — 20 쪽
6. Damager 의 Hittable Layers 를 비워 두면 어떻게 되나? — 12 · 22 쪽
7. Chomper와 Spitter는 같은 컴포넌트를 쓴다. 무엇이 둘을 다르게 만드나? — 11 · 23 쪽
8. 총알을 Instantiate / Destroy 로 만들면 무엇이 문제인가? — 24 쪽
9. 움직이는 플랫폼 위에서 점프가 되는 이유는? 왜 자식으로 붙이지 않나? — 25 쪽
10. 씬을 넘어갈 때 저장은 페이드 전인가 후인가? 왜 그 순서인가? — 27 쪽
11. 행동 트리에서 Continue 가 있기 때문에 가능해지는 것은 무엇인가? — 28 쪽
12. 이 키트의 코드에서 "결함"이라고 부를 만한 것을 두 가지 들어 보라 — 32 쪽

126개 스크립트를 다 읽을 필요는 없다. 필요할 때 어디를 열면 되는지 알면 된다.

Copyright© by Wonkwang DC